📘 Manuale Utente — La Mia Configurazione Neovim

> Guida pratica, completa e definitiva a come si usa **questa** configurazione di Neovim, basata su LazyVim. Non è un manuale generico su Neovim: ogni tasto, ogni comportamento e ogni plugin descritto qui è esattamente quello che trovi aprendo questo editor, oggi, così com'è. L'obiettivo è uno solo: che tu sappia **cosa fare, quando farlo e perché**, senza dover indovinare nulla.

Indice

1. Filosofia e Fondamenta
2. La Schermata Iniziale e l'Interfaccia
3. Navigare il Progetto: Explorer e Picker
4. Il Terminale Integrato
5. Sviluppo Guidato dall'LSP: Python, TypeScript e non solo
6. Diagnostica Discreta e Trouble.nvim
7. Completamento Intelligente: blink.cmp e Codeium
8. Manutenzione del Sistema

---

1. Filosofia e Fondamenta

Il tasto Leader: il tuo "prefisso personale"

Quasi ogni funzione avanzata di questa configurazione si raggiunge premendo prima la **barra spaziatrice**, poi una lettera (o una sequenza di lettere). Questa combinazione si chiama, nel gergo di Vim/Neovim, il tasto `<leader>`, ed è mappato sulla **barra spaziatrice**.

Quando in questo manuale leggi `<leader>ff`, significa letteralmente: *premi Spazio, rilascialo, poi premi f e poi f di nuovo*. Non è una combinazione simultanea (come `Ctrl+S`), è una **sequenza**: premi un tasto dopo l'altro, con calma. Esiste anche un tasto `<localleader>`, mappato sul backslash (`\`), usato da pochissime funzioni specifiche di un linguaggio (per esempio l'esecuzione rapida di codice Lua).

Non devi imparare a memoria le centinaia di combinazioni disponibili: basta premere **Spazio** e **aspettare mezzo secondo senza premere altro**. Comparirà automaticamente un popup (fornito da **which-key**) con tutti i gruppi di comandi disponibili, organizzati per categoria: `f` per file, `s` per search (ricerca), `g` per git, `c` per code, `x` per diagnostica/quickfix, `b` per buffer, `w` per finestre, `u` per

interfaccia/toggle. Premendo la lettera del gruppo che ti interessa, il popup si aggiorna mostrando i comandi disponibili in quel sottogruppo. È il modo più naturale per *scoprire* le funzionalità mentre lavori, invece di studiarle a memoria.

Le modalità: il cuore del funzionamento di Neovim

Neovim non funziona come un editor di testo "normale" in cui digiti e basta. Funziona a **modalità**: lo stesso tasto fa cose completamente diverse a seconda di dove ti trovi. Capire questo concetto è la chiave per usare bene tutto il resto della configurazione.

Modalità	Come ci entri	Cosa puoi fare
Normal	Premi `<Esc>` da qualunque altra modalità	È la modalità "di comando": qui ti muovi, cancelli, copi, incolli, lanci comandi. **Non stai scrivendo testo.** Ogni sessione di editing parte e torna sempre qui.
Insert	`i` (inserisci prima del cursore), `a` (dopo), `o` / `O` (nuova riga sotto/sopra)	Qui digiti testo come in un editor qualsiasi. È l'unica modalità in cui premere una lettera la scrive nel file.
Visual	`v` (seleziona carattere per carattere), `V` (seleziona righe intere), `<C-v>` (seleziona un blocco rettangolare)	Selezioni del testo per poi agire su di esso: cancellarlo, copiarlo, indentarlo, farci una sostituzione, commentarlo.
Command-line	`:` dalla Normal mode	Digiti comandi "Ex": `:w` per salvare, `:%s/vecchio/nuovo/g` per una sostituzione, `:Mason` per aprire un plugin.
Terminal	Aprendo un terminale integrato (vedi Sezione 4)	Un vero terminale di sistema dentro un buffer di Neovim, con la sua modalità Insert/Normal dedicata.

La cosa più importante da interiorizzare: **`<Esc>` ti riporta sempre alla base (Normal mode)**, da qualunque punto tu sia. Se ti perdi, non sai più in che modalità sei, o qualcosa si comporta in modo strano: premi `<Esc>` una o due volte e sei di nuovo a "casa". In questa configurazione, premere `<Esc>` in Normal, Insert o Select mode fa anche un'altra cosa utile: **spegne l'evidenziazione della ricerca** (`hlsearch`) se ne avevi appena fatta una — un modo elegante per "pulire lo schermo" con un solo tasto.

Muoversi nel testo

Prima di modificare qualcosa, devi potertici mettere sopra al volo. Oltre alle frecce, in Normal mode hai a disposizione questi movimenti (che tornano utilissimi anche nella sezione successiva):

Tasto	Effetto
`h` / `j` / `k` / `l`	sinistra / giù / su / destra (le "frecce" storiche di Vim, sotto le dita senza spostare la mano)
`w`	Salta all'inizio della **prossima parola**
`b`	Torna all'inizio della parola **precedente**
`e`	Salta alla **fine** della parola corrente/successiva
`0`	Inizio della riga corrente
`$`	Fine della riga corrente
`gg`	Inizio del file
`G`	Fine del file

Puoi anche far precedere quasi ogni movimento da un numero per ripeterlo: `3w` salta avanti di tre parole, `5j` scende di cinque righe. Questo stesso trucco (numero + comando) funziona anche con i comandi di modifica che vedi qui sotto.

Modificare il testo: eliminare, copiare, incollare, sostituire

Qui arriva il concetto che rende Vim/Neovim così rapido da usare una volta presa la mano: quasi tutti i comandi di modifica seguono la stessa logica, **azione + bersaglio**. Il "bersaglio" è quasi sempre uno dei movimenti che hai appena visto sopra (`w` , `$` , `gg` ...), oppure una riga intera. Impari poche azioni, e le puoi combinare con qualsiasi movimento già sai.

Un'altra cosa fondamentale da sapere subito: **in Vim non esiste un comando "taglia" separato da "cancella"**. Ogni volta che cancelli qualcosa (un carattere, una parola, una riga), quel testo finisce automaticamente in una specie di "appunti" interno, pronto per essere incollato altrove — è già, di fatto, un taglia-e-incolla. Per questo i comandi di cancellazione e quelli di taglia sono gli stessi.

Eliminare un carattere

Tasto	Effetto
`x`	Cancella il carattere **sotto** il cursore
`X`	Cancella il carattere **prima** del cursore (come il tasto Backspace)

Esempio: hai scritto `ciaoo` per errore (una "o" di troppo alla fine) e il cursore è sull'ultima lettera: premi `x` una volta e diventa `ciao` .

Eliminare intere righe

Tasto	Effetto
`dd`	Cancella **l'intera riga** su cui si trova il cursore
`3dd` (numero + `dd`)	Cancella 3 righe a partire da quella corrente
`d$`	Cancella dal cursore fino alla **fine della riga** (utile se vuoi tenere l'inizio della riga e buttare solo il resto)
`dw`	Cancella dal cursore fino all'inizio della **prossima parola**

Esempio: hai una riga inutile in mezzo al file (magari un `console.log` di debug dimenticato): metti il cursore su quella riga, premi `dd`, e sparisce del tutto — le righe sotto risalgono di una posizione, come ci si aspetta.

Copiare e incollare (anche tagliare)

Tasto	Effetto
`yy`	Copia ("yank") l'intera riga corrente, **senza cancellarla**
`3yy`	Copia 3 righe a partire da quella corrente
`p`	Incolla **dopo** la riga corrente (o dopo il cursore, se stai incollando testo copiato a metà riga)
`P`	Incolla **prima** della riga corrente (maiuscola = sopra)

Come detto sopra, per "tagliare" non serve un tasto diverso: usi semplicemente `dd` (o `x`, `dw`...) per cancellare, e poi `p` / `P` per incollare altrove — il testo cancellato è già pronto in memoria.

Esempio — spostare una riga più in basso nel file: metti il cursore sulla riga da spostare, premi `dd` (sparisce da lì), scendi con `j` fino alla riga sotto cui la vuoi ricollocare, premi `p`: la riga ricompare esattamente lì. **Esempio — duplicare una riga**: `yy` per copiarla senza cancellarla, poi `p` per incollarne una copia subito sotto: ora ne hai due identiche, una sopra l'altra.

Per copiare/tagliare un **blocco di testo più ampio e arbitrario** (non righe intere, ma un pezzo di frase, o più righe non contigue con un confine preciso), usi la modalità **Visual** vista in precedenza: premi `v` (o `V` per selezionare righe intere), muovi il cursore per estendere la selezione evidenziata, poi premi `y` per copiarla o `d` per cancellarla/tagliarla — infine `p` per incollarla dove vuoi, come sopra.

Esempio: vuoi spostare tre righe di codice non consecutive alla fine, ma solo una parte specifica di un blocco più grande. Premi `V` sulla prima riga da prendere, scendi con `j` fino a includere anche le altre due (le vedi evidenziate man mano), premi `d` per tagliarle tutte insieme, spostati dove vuoi incollarle, premi `p`.

Sostituire e sovrascrivere

Tasto	Effetto
`r` + un carattere	Sostituisce **un solo carattere** sotto il cursore con quello che digiti subito dopo, senza mai entrare in modalità Insert
`cw`	Cancella la parola corrente e ti mette **subito in Insert mode**, pronto a scriverne una nuova al suo posto (il resto della riga resta intatto)
`cc` (o `S`)	Cancella **l'intera riga** e ti mette in Insert mode per riscriverla da zero, mantenendo l'indentazione
`R`	Entra in modalità **Replace**: da qui in poi, ogni carattere che digiti **sovrascrive** quello già presente invece di spingerlo in avanti (utile per correggere un refuso mantenendo la stessa lunghezza di riga); premi `<Esc>` per uscirne come da una normale Insert mode

Esempio: hai scritto `calcolaTotle` invece di `calcolaTotale` — anziché cancellare tutto e riscrivere, metti il cursore sulla parola e premi `cw`, digiti `calcolaTotale` e premi `<Esc>`: fatto, solo quella parola è cambiata. **Esempio con `r`**: hai scritto `ciaoo` (di nuovo la "o" di troppo, ma stavolta vuoi sostituirla con una "!"): cursore sull'ultima lettera, premi `r` poi `!`: diventa `ciao!` in un solo gesto, senza mai entrare in modalità Insert.

Annullare, ripetere, e il trucco del "ripeti l'ultima modifica"

Tasto	Effetto
`u`	Annulla (undo) l'ultima modifica
`<C-r>`	Ripristina (redo) ciò che avevi appena annullato
`.` (punto)	**Ripete l'ultima modifica** fatta, qualunque essa sia

Il tasto `.` è uno dei più potenti e sottovalutati di Vim: qualsiasi cosa tu abbia appena fatto — cancellato una parola con `dw`, sostituito un carattere con `r`, cancellato una riga con `dd` — premendo `.` la **rifai identica, sul punto in cui ti trovi ora**, senza dover ripetere la sequenza di tasti.

Esempio: hai tre righe diverse, ciascuna con una parola da cancellare con `dw`. La cancelli sulla prima riga, scendi con `j` sulla seconda, premi `.`: la stessa cancellazione si ripete lì. Scendi ancora, premi `.` di nuovo: fatto anche sulla terza. Tre modifiche identiche, un solo comando imparato.

Salvare e uscire

Comando	Dove funziona	Effetto
`<C-s>`	Insert, Normal, Visual	Salva il file **e** ti riporta in Normal mode, in un solo colpo. È il modo più comodo: puoi premerlo anche mentre stai scrivendo, senza dover uscire prima dalla modalità Insert.
`:w`	Command-line	Salva, alla vecchia maniera.
`:q`	Command-line	Chiude la finestra corrente (se ci sono altre finestre/buffer aperti, Neovim resta acceso).
`:wq` oppure `:x`	Command-line	Salva ed esce dalla finestra corrente.
`<leader>qq`	Normal	Chiude **tutto** Neovim (tutte le finestre, tutte le tab).

Se provi a chiudere un buffer con modifiche non salvate, non otterrai un errore secco: Neovim ti **chiede conferma** ("vuoi salvare, scartare o annullare?"), esattamente come un editor grafico moderno.

Buffer: i tuoi file aperti "in memoria"

Ogni file che apri in Neovim diventa un **buffer**: una copia del contenuto tenuta in memoria, che puoi modificare, salvare, o tenere aperta anche senza che sia visibile a schermo in quel momento. Puoi avere decine di buffer aperti contemporaneamente, anche se ne visualizzi solo uno o due.

In cima allo schermo trovi sempre una barra dei tab (fornita da **bufferline**) con l'elenco dei buffer aperti, ciascuno con la sua icona in base al tipo di file. Passare da un buffer all'altro è immediato:

Tasto	Azione
`<S-h>` / `<S-l>` (Shift+h / Shift+l)	Buffer precedente / successivo
`[b` / `]b`	Stesso effetto, alternativa in stile "testo precedente/successivo"
`<leader>fb`	Apri un **picker** con l'elenco di tutti i buffer aperti (utile quando ne hai tanti e vuoi saltare direttamente a uno specifico digitandone il nome, invece di ciclare uno a uno)
`<leader>bd`	Chiudi **il buffer corrente**, senza chiudere la finestra né toccare gli altri
`<leader>bo`	Chiudi **tutti gli altri** buffer, tenendo aperto solo quello corrente
`<leader>bp`	"Fissa" (pin) il buffer corrente nella barra dei tab, così resta sempre visibile anche quando ne apri molti altri

Caso d'uso pratico: stai lavorando su tre file collegati (un componente, il suo test, e un file di stile). Li apri tutti e tre: ora sono tre buffer, visibili come tab in cima allo schermo. Per passare velocemente dall'uno all'altro: `<S-l>` / `<S-h>` per ciclare avanti/indietro senza mai passare dal file-explorer, oppure `<leader>fb` se preferisci vederli tutti in un elenco e scegliere direttamente quello che ti serve. Quando hai finito con uno e vuoi **chiuderlo**: portati sopra con `<S-l>` / `<S-h>` (o selezionalo dall'elenco) e premi

`<leader>bd` — si chiude solo quello, gli altri due restano aperti esattamente come prima. Se invece a un certo punto vuoi tenere aperto solo il file su cui stai lavorando ora e liberarti di tutto il resto in un colpo solo, usa `<leader>bo`.

Finestre e split: vedere più cose insieme

Un buffer viene *mostrato* dentro una **finestra** (window). Puoi dividere lo schermo in più finestre per vedere due (o più) buffer contemporaneamente, affiancati o impilati.

Tasto	Azione
`<leader>-`	Divide la finestra corrente **orizzontalmente** (una sopra, una sotto)
`<leader>|`	Divide la finestra corrente **verticalmente** (una a sinistra, una a destra)
`<C-h>` / `<C-j>` / `<C-k>` / `<C-l>`	Sposta il focus tra le finestre, nella direzione indicata (sinistra/giù/su/destra) — esattamente come muoverti con `hjkl`, ma tra finestre invece che tra caratteri
`<C-Up>` / `<C-Down>` / `<C-Left>` / `<C-Right>`	Ridimensiona la finestra attiva (più alta/bassa/stretta/larga)
`<leader>wd`	Chiude la finestra corrente (il buffer resta comunque aperto altrove, se visibile)
`<leader>wm`	"Zoom": ingrandisce la finestra corrente a schermo intero temporaneamente, premuto di nuovo torna al layout precedente

Caso d'uso pratico: stai scrivendo una funzione e vuoi tenere sott'occhio la sua documentazione o il file che la chiama. `<leader>|` apre uno split verticale, `<C-l>` ci sposti il focus, apri il secondo file lì dentro, e con `<C-h>` / `<C-l>` salti avanti e indietro tra i due mentre scrivi. Lo stesso identico meccanismo (`<C-h/j/k/l>`) funziona anche per spostarti dentro e fuori dal terminale integrato (Sezione 4) e dall'albero dei file (Sezione 3): è la logica di navigazione universale di tutta questa configurazione.

---

2. La Schermata Iniziale e l'Interfaccia

La Dashboard

Quando apri Neovim senza specificare un file (semplicemente digitando `nvim` nel terminale), non ti ritrovi davanti a uno schermo vuoto: ti accoglie la **Dashboard**, una schermata con un grande banner ASCII "LAZYVIM" e un menu di azioni rapide, ciascuna richiamabile con una singola lettera:

Tasto sulla Dashboard	Azione
f	Trova un file nel progetto (apre il Picker)
n	Crea un nuovo file vuoto e ti mette subito in modalità scrittura
g	Cerca del testo in tutto il progetto (grep)
r	Mostra i file aperti di recente
c	Apri un file di configurazione di Neovim (utile per modificare questi stessi file)
s	Ripristina l'ultima sessione di lavoro salvata
p	Elenco dei "progetti" riconosciuti (cartelle in cui hai già lavorato)
x	Apre `:LazyExtras`, il pannello per abilitare/disabilitare intere funzionalità linguistiche
l	Apre `:Lazy`, il gestore dei plugin
q	Esce da Neovim

Non devi per forza chiudere Neovim per tornare a questa schermata: `<leader>h` la riapre in qualsiasi momento, anche a metà lavoro con dieci file già aperti — utile quando vuoi "ripartire da zero" senza chiudere la sessione, o semplicemente per usare la ricerca file/testo direttamente da lì.

La barra di stato (Lualine)

In fondo allo schermo trovi sempre una barra di stato, che a colpo d'occhio ti dice tutto quello che ti serve sapere sul momento:

- **A sinistra**: la modalità in cui ti trovi (NORMAL, INSERT, VISUAL...) colorata in modo diverso per riconoscerla al volo, il branch git corrente, la cartella radice del progetto.
- **Al centro**: eventuali errori/avvisi di diagnostica presenti nel buffer (con conteggio), l'icona del tipo di file, il percorso del file corrente (abbreviato in modo intelligente se è lungo).
- **A destra**: lo stato del completamento AI di Codeium, eventuali aggiornamenti di plugin disponibili, le modifiche git non ancora committate (righe aggiunte/modificate/rimosse), la posizione nel file, e — per un tocco di leggerezza — un'emoji 😄 che chiude la barra.

Non è decorazione fine a sé stessa: è pensata per non doverti mai fermare a chiedere "in che file sono, su che branch, ci sono errori?" — è tutto già lì, sempre visibile.

I messaggi e la command-line (Noice)

Quando digiti `:` per lanciare un comando, non vedi la classica riga in fondo allo schermo: ti compare un **popup elegante** al centro, con evidenziazione della sintassi del comando che stai scrivendo. Allo stesso modo, notifiche, messaggi di errore e output di comandi non ingombrano la parte bassa dello schermo ma appaiono come popup temporanei, più leggibili e meno invasivi.

Se un messaggio scompare troppo in fretta e vuoi rileggerlo:

Comando	Azione
`:Noice last` (o `<leader>snl`)	Riapre l'ultimo messaggio mostrato
`:Noice history` (o `<leader>snh`)	Apre la cronologia completa dei messaggi
`<leader>snd`	Chiude/pulisce tutte le notifiche a schermo

Il tema: Catppuccin

Il colore di tutta l'interfaccia è **Catppuccin**, nella variante scura *Mocha*. Se in un dato momento (magari lavorando di giorno, alla luce) preferisci qualcosa di diverso, non serve modificare nessun file: premi `<leader>uC` per aprire un selettore di temi **con anteprima dal vivo** — puoi scorrere tutti i colorscheme installati e vedere l'effetto in tempo reale prima di confermare con Invio (annulla con `<Esc>` per tornare al tema di partenza).

---

3. Navigare il Progetto: Explorer e Picker

Questa configurazione usa **Snacks** come motore sia per l'albero dei file sia per la ricerca rapida. Sono due strumenti diversi per due bisogni diversi: l'**Explorer** serve quando vuoi vedere e manipolare la struttura delle cartelle; il **Picker** serve quando sai (più o meno) cosa stai cercando — un nome di file, una parola nel codice — e vuoi arrivarci nel minor numero di tasti possibile, senza mai staccare le mani dalla tastiera.

L'albero dei file (Explorer)

Tasto	Azione
`<leader>e`	Apre l'albero dei file, radicato nella cartella del progetto
`<leader>E`	Apre l'albero, radicato nella cartella corrente (utile se stai lavorando in una sotto-cartella specifica)

Una volta dentro l'albero, ti muovi con `j` / `k` (o le frecce) per scorrere le voci, e usi questi tasti per operare sui file:

Tasto	Azione
Invio / `l`	Apre il file selezionato (in una finestra), oppure espande/richiude una cartella
`h`	Richiude una cartella espansa, o sale al livello superiore
`a`	Crea un nuovo file o cartella nel punto selezionato (digita il nome; termina con `/` per creare una cartella)
`d`	Elimina il file o la cartella selezionata (con richiesta di conferma)
`r`	Rinomina il file/cartella selezionata
`y` / `x` / `p`	Copia / taglia / incolla il file selezionato
`g?`	Mostra l'elenco completo di tutti i comandi disponibili nell'albero, nel caso ti serva qualcosa di più avanzato

Caso d'uso pratico: stai creando un nuovo componente e vuoi che finisca dentro `src/components/`. Apri l'albero con `<leader>e`, scorri fino alla cartella `components`, premi `a`, digiti `Bottone.tsx` e premi Invio: il file viene creato al posto giusto e si apre subito, pronto per scriverci dentro. Se poi ti accorgi di aver sbagliato nome, torni sull'albero, selezioni il file, premi `r`, correggi e confermi: Neovim si accorge da solo che il buffer aperto corrisponde a quel file e lo tiene sincronizzato.

Trovare file e testo (Picker)

Il Picker è lo strumento che userai più spesso in assoluto, perché risponde alla domanda più comune quando lavori su un progetto: "*so che questa cosa esiste da qualche parte, ma non ricordo esattamente dove*".

Tasto	Azione	Quando usarlo
`<leader><space>` o `<leader>ff`	Trova un file per nome, in tutto il progetto	Sai (anche solo vagamente) come si chiama il file che ti serve
`<leader>fr`	Elenco dei file aperti di recente	Stavi lavorando su qualcosa ieri e vuoi ripartire da lì
`<leader>fg`	Trova tra i soli file tracciati da git	Vuoi escludere `node_modules`, file generati, ecc. senza doverli filtrare a mano
`<leader>fb`	Elenco dei buffer già aperti	Hai già aperto il file, vuoi solo saltarci sopra
`<leader>/` o `<leader>sg`	**Grep**: cerca una stringa di testo in tutti i file del progetto	Non sai in che file si trova qualcosa, ma sai cosa c'è scritto
`<leader>sw`	Grep della parola esatta sotto il cursore	Vuoi vedere subito *tutti* i posti in cui compare un nome, senza doverlo ridigitare
`<leader>sb`	Cerca solo nelle righe del buffer aperto in questo momento	Il file è lungo e vuoi saltare a una riga specifica senza scorrere a mano

In ogni caso, il funzionamento è lo stesso: si apre una finestra con un campo di ricerca in alto e i risultati sotto, **aggiornati in tempo reale a ogni carattere digitato** (fuzzy-matching: non serve scrivere il nome esatto, bastano le lettere giuste nell'ordine giusto). Ti muovi tra i risultati con le frecce o `<C-n>` / `<C-p>`, e premi Invio per aprire quello selezionato. Nella maggior parte dei picker, una finestra di anteprima a fianco ti mostra il contenuto del file/la riga trovata **prima ancora di aprirlo**, cosa che ti risparmia un sacco di "apri, controlla, richiudi" a vuoto.

Caso d'uso pratico n.1 — cercare un file: vuoi modificare il componente che gestisce il login. Premi `<leader>ff`, digiti `login`: nel giro di una frazione di secondo vedi tutti i file il cui percorso contiene quella sequenza di lettere (`LoginForm.tsx`, `useLogin.ts`, `login.test.ts`...). Frecce su/giù per scegliere, Invio per aprire.

Caso d'uso pratico n.2 — trovare tutti gli usi di una funzione, ovunque: hai una funzione chiamata `formatCurrency` e vuoi sapere in quanti punti del progetto viene usata, **inclusi commenti, file di configurazione o README** che l'LSP da solo non vedrebbe mai (l'LSP conosce solo il codice del linguaggio che sta analizzando, non file `.md` o `.json`). Metti il cursore sul nome della funzione e premi `<leader>sw`: ottieni l'elenco completo di ogni riga di ogni file in cui compare quella stringa esatta, in tutto il repository.

Caso d'uso pratico n.3 — "ho in mente delle parole ma non so dove": stai cercando dove viene gestito un certo messaggio di errore che appare a schermo nell'app. Premi `<leader>sg`, digiti un pezzo del testo del messaggio: il Picker cerca quella stringa in ogni file di progetto e te ne mostra ogni occorrenza, con anteprima.

---

4. Il Terminale Integrato

Uno dei vantaggi di lavorare con l'editor invece di tenere un terminale separato è poterne aprire uno **dentro** Neovim stesso, senza perdere il contesto di cosa stavi facendo. Questa configurazione offre diversi modi per farlo, a seconda di cosa ti serve.

Aprire un terminale

Tasto	Cosa apre	Tipo di finestra
`<leader>ft`	Un terminale posizionato nella **cartella radice del progetto**	Si apre come **split in basso**, largo quanto lo schermo
`<leader>fT`	Un terminale nella cartella corrente (utile se sei dentro una sotto-cartella e vuoi restarci)	Split in basso
`<C-t>`	Apre/riporta a fuoco lo stesso terminale della root — funziona da qualunque punto, anche da dentro il terminale stesso (vedi sotto)	Split in basso
`<leader>gg`	Apre **Lazygit**, l'interfaccia testuale per Git (staging, commit, branch, log, tutto a portata di tasto)	**Finestra flottante** (popup), centrata sopra il codice

Passare velocemente dal codice al terminale (e viceversa)

Questo è il gesto che userai più spesso in assoluto, quindi vale la pena capirlo bene: `<C-t>` (Ctrl+t) è un vero e proprio **interruttore avanti-indietro**, e funziona da qualsiasi punto tu sia, in un solo tasto:

- Sei nel codice e vuoi passare al terminale → premi `<C-t>`: ti sposta subito lì (lo crea se non è già aperto).
- Sei dentro il terminale — **anche mentre stai scrivendo un comando, in piena modalità Insert** — e vuoi tornare al codice → premi di nuovo `<C-t>`: ti riporta istantaneamente alla finestra di prima.

Non serve mai uscire prima dalla modalità Insert per usarlo: funziona identico in entrambe le direzioni, con lo stesso identico tasto. Il processo dentro il terminale resta sempre vivo in sottofondo mentre sei nel codice, pronto a riprendere esattamente da dove l'avevi lasciato.

Caso d'uso pratico: stai scrivendo codice e vuoi lanciare al volo `npm run test` senza smettere di guardare il file aperto. Premi `<C-t>`: si apre lo split in basso, lanci il comando, guardi l'output. Premi di nuovo `<C-t>`: il terminale scompare e recuperi tutto lo schermo per il codice, ma il processo (se è ancora in esecuzione, es. un watcher) continua a girare in sottofondo. Quando vuoi ricontrollare l'output, `<C-t>` lo fa ricomparire esattamente com'era.

Uscire dalla modalità Insert restando nel terminale

A volte non vuoi tornare al codice: vuoi restare fermo *dentro* il terminale, ma smettere di inviargli tasti — per esempio per scorrere l'output con calma, selezionare del testo da copiare, o chiuderlo definitivamente (vedi sotto). In questo caso, invece di `<C-t>`, usi una combinazione che ti fa uscire dalla modalità **Terminal-Insert** e passare alla **Terminal-Normal**, restando nella stessa finestra:

Combinazione	Effetto
`<Esc>` `<Esc>` (doppio Escape, premuto due volte in rapida sequenza)	Esce dalla modalità Insert del terminale e torna in modalità Normal, **sul buffer del terminale stesso**
`jk` (premuto in rapida sequenza)	Stesso identico effetto, alternativa comoda per chi preferisce non staccare le dita dalla riga home della tastiera, e più affidabile perché non dipende da un tempismo preciso

Perché il doppio Escape e non uno singolo? Perché un singolo `<Esc>` dentro un terminale ha un suo significato legittimo per moltissimi programmi a riga di comando che potresti lanciarci dentro (per esempio un altro editor, un menu interattivo, un debugger): quel primo `<Esc>` va quindi comunque inviato al programma in esecuzione. È solo il **secondo** `<Esc>`, premuto entro una frazione di secondo dal primo, a dire a Neovim "ok, adesso voglio comandare te, non il programma dentro il terminale". Se le due pressioni non sono abbastanza ravvicinate, vengono interpretate come due Escape separati inviati al programma, e resti in Insert mode: se hai un dubbio, usa `jk`, che non ha questo problema.

Una volta in modalità Normal sul terminale, puoi:

- Scorrere l'output con `j` / `k`, `<C-u>` / `<C-d>` come in un buffer normale.

- Spostarti su un'altra finestra con `<C-h>` / `<C-j>` / `<C-k>` / `<C-l>` — esattamente gli stessi tasti che usi per muoverti tra split di codice (Sezione 1): il terminale, agli occhi della navigazione tra finestre, è una finestra come le altre.
- Rientrare in modalità Insert (per continuare a digitare comandi nella shell) semplicemente premendo `i` o `a`, come in un buffer di testo qualsiasi.

Caso d'uso pratico: hai lanciato un comando che produce molto output e vuoi rileggerlo con calma scorrendo in su. Premi `jk` per uscire dalla modalità Insert, scorri con `k` / `<C-u>`, poi premi `<C-t>` per tornare al tuo file di codice senza nemmeno chiudere il terminale, che resta lì pronto quando ti servirà di nuovo.

Chiudere definitivamente il terminale

C'è una differenza importante tra **nascondere** un terminale e **chiuderlo per sempre**: `<C-t>` (e anche `<leader>ft` / `<leader>fT` premuti su uno già aperto) lo nascondono soltanto — il processo resta vivo in background, così puoi farlo ricomparire esattamente com'era. Se invece vuoi eliminare davvero quello split (per esempio perché il comando è finito e non ti serve più tenerlo in memoria), usa:

Tasto	Dove funziona	Effetto
`<C-q>`	Sia da dentro il terminale (anche in Insert mode) sia da qualunque altra finestra con il focus sopra di esso	Esce dalla modalità Insert **e** chiude definitivamente lo split, in un solo tasto

`<C-q>` è pensato apposta per non doverti preoccupare di "in che modalità sono": funziona comunque, sempre, con un solo gesto.

---

5. Sviluppo Guidato dall'LSP: Python, TypeScript e non solo

Ogni linguaggio configurato (Python, TypeScript/JavaScript, Lua, C/C++, Java, LaTeX...) è collegato a un **LSP** (Language Server Protocol): un programma esterno che analizza il tuo codice in tempo reale e fornisce a Neovim autocompletamento consapevole dei tipi, la possibilità di saltare a una definizione, rinominare in sicurezza, e molto altro. La bellezza del sistema è che **gli stessi tasti funzionano identici in ogni linguaggio**: impari una volta, usi ovunque.

Il flusso di lavoro universale (vale per qualsiasi linguaggio con LSP attivo)

Tasto	Cosa fa, e quando usarlo
`K`	**Hover**: mostra al volo, in un popup, la documentazione/firma di ciò che sta sotto il cursore (una funzione, una variabile, un tipo). Usalo ogni volta che incontri qualcosa di cui non ricordi esattamente la firma o il tipo di ritorno — niente più saltare in un altro file per controllare.
`gd`	**Vai alla definizione**: salta esattamente al punto in cui una funzione/variabile/classe è stata definita, anche se si trova in un altro file.
`gr`	**Trova i riferimenti**: mostra (in un Picker) ogni punto del progetto in cui il simbolo sotto il cursore viene usato. Utilissimo prima di modificare qualcosa, per capire quanto "impatto" avrà la modifica.
`gI`	Vai all'implementazione (utile con interfacce/classi astratte, per saltare all'implementazione concreta)
`<leader>ca`	**Code action**: mostra un menu contestuale di "cose intelligenti che l'LSP può fare per te qui" — puo trattarsi di estrarre una variabile, aggiungere un cast, implementare un'interfaccia mancante, e molto altro; il contenuto cambia in base al linguaggio e al punto esatto del codice.
`<leader>cr`	**Rinomina il simbolo**: rinomina una variabile, funzione o classe **in ogni punto del progetto in cui viene usata**, non solo nel file corrente. Digiti il nuovo nome una volta sola, premi Invio, e l'LSP aggiorna atomicamente ogni singolo riferimento in tutti i file coinvolti — cosa che un semplice "trova e sostituisci" testuale non potrebbe fare con la stessa sicurezza (perché capisce che si tratta dello *stesso* simbolo, e non di un'altra variabile che si chiama per caso allo stesso modo).
`<leader>cR`	**Rinomina il file** corrente, aggiornando automaticamente tutti gli `import` / `require` che lo referenziano altrove nel progetto. Preziosissimo quando riorganizzi la struttura delle cartelle: non devi più andare a caccia manualmente di ogni file che importava quello spostato.
`<leader>co`	Organizza automaticamente gli import del file (li ordina, rimuove quelli inutilizzati, se il linguaggio lo supporta)
`<leader>cf`	Formatta l'intero buffer secondo le regole del linguaggio
`]d` / `[d`	Salta alla prossima/precedente diagnostica (errore o avviso) nel buffer

Caso d'uso pratico — rinominare in sicurezza: hai una funzione `calcolaTotale` usata in dodici punti diversi del progetto e ti sei accorto che il nome non è più adatto. Ti metti sopra il nome con il cursore, premi `<leader>cr`, digiti `calcolaTotaleConSconto`, premi Invio: tutti e dodici i punti vengono aggiornati all'istante, senza rischio di aver dimenticato un'occorrenza o di aver rinominato per sbaglio un'altra variabile omonima appartenente a un contesto diverso.

Python: Pyright, Ruff e la gestione dei virtual environment

Su un file `.py` sono al lavoro **due strumenti in parallelo**, ciascuno con un compito preciso:

- **Pyright** si occupa di tutto ciò che riguarda i **tipi**: autocompletamento consapevole delle classi/funzioni disponibili, `K` per l'hover con firme e tipi, `gd` per saltare alle definizioni, il controllo dei tipi in generale.
- **Ruff** si occupa di **linting velocissimo**: ti segnala import inutilizzati, variabili mai usate, stile del codice non conforme, e in molti casi può **correggere automaticamente** il problema tramite `<leader>ca` (compare come una delle code action proposte). Non fa da doppione con Pyright per l'hover: quel compito è lasciato solo a Pyright, così non ti ritrovi due popup di documentazione sovrapposti.

Il punto più delicato quando si lavora in Python è che l'ambiente virtuale (**virtualenv/venv**) giusto per il progetto potrebbe non essere quello di sistema: se l'LSP analizza il codice con l'interprete Python sbagliato, ti segnalerà falsi errori del tipo "modulo non trovato" per ogni libreria installata nel tuo venv (che l'LSP, semplicemente, non sa che esiste). Per questo:

Tasto	Azione
`<leader>cv`	Apre un selettore (**VenvSelect**) che elenca i virtual environment rilevati nel progetto (o sul sistema): scegli quello giusto e Pyright ricomincia ad analizzare il codice con l'interprete e le librerie corrette

Caso d'uso pratico: apri un progetto Python clonato da poco, e Pyright ti segnala in rosso `import pandas` come "modulo non risolto", anche se sai per certo di averlo installato. Premi `<leader>cv`, selezioni il venv del progetto (es. `.venv`) dall'elenco, e gli errori spariscono: Pyright ora sa dove cercare i pacchetti installati.

TypeScript e JavaScript: vtsls

Il motore che analizza i file `.ts` / `.tsx` / `.js` / `.jsx` è **vtsls**, e oltre ai comandi universali descritti sopra (hover, goto definition, rename, code action...) mette a disposizione qualche scorciatoia specifica, pensata per i problemi tipici di un progetto TypeScript reale:

Tasto	Azione	Quando ti serve davvero
`gD`	Vai alla definizione "**sorgente**" (source definition)	Quando `gd` ti porta su un file di *re-export* (`export { X } from './altrofile'`) invece che sulla vera definizione: questo comando salta dritto all'origine reale, saltando gli intermediari
`gR`	Trova **tutti i riferimenti al file** corrente (non a un simbolo: proprio al file nel suo insieme)	Prima di spostare o eliminare un file, per capire chi lo importa
`<leader>cM`	**Aggiungi gli import mancanti**	Hai incollato del codice che usa una funzione/tipo non ancora importato in questo file: un tasto e l'import compare in cima, senza doverlo scrivere a mano
`<leader>cD`	**Correggi automaticamente tutte le diagnostiche risolvibili** nel buffer	Fine di una sessione di scrittura veloce: un colpo solo per ripulire piccoli problemi (import, punti e virgola, banalità di tipo) invece di correggerli uno a uno
`<leader>cV`	Seleziona quale versione di TypeScript usare per l'analisi	Utile in monorepo con più `package.json`/versioni di TypeScript, quando l'editor sceglie quella "sbagliata" per la cartella in cui ti trovi

Caso d'uso pratico: stai copiando una funzione di utilità da un altro file del progetto dentro quello che stai scrivendo. Appena incollata, vtsls la sottolinea perché il `import` non esiste ancora in questo file. Invece di scrivere a mano il percorso relativo giusto, premi `<leader>cM`: l'import corretto compare automaticamente in cima al file.

---

6. Diagnostica Discreta e Trouble.nvim

Questa configurazione fa una scelta precisa in fatto di errori e avvisi: **niente testo lungo ed invasivo a fine riga**. Molti editor, quando c'è un errore, scrivono un messaggio esteso proprio accanto al codice, spostando tutto e rendendo il file illeggibile quando ci sono più problemi vicini. Qui non succede: la segnalazione è volutamente **discreta**, ma non per questo meno completa.

Come ti accorgi che c'è un problema

- **Un'iconcina colorata nel margine sinistro** (la colonna dei segni), diversa per errore/avviso/info/hint: la vedi con la coda dell'occhio scorrendo il file, senza che il testo si sposti di un millimetro.
- **Una sottolineatura ondulata** proprio sotto il tratto di codice incriminato — sai *esattamente* quale porzione è in causa, senza dover indovinare a quale parola si riferisce un messaggio scritto altrove.

Come leggi il dettaglio del messaggio

La segnalazione discreta non significa che il messaggio sparisce: significa solo che **lo richiami tu, quando vuoi leggerlo**, invece di doverlo subire sempre a schermo.

Tasto	Azione
`K`	Se il cursore è su una riga con un problema, l'hover mostra (insieme all'eventuale documentazione) anche il messaggio di diagnostica completo
`<leader>cd`	Apre un popup con il dettaglio di ogni diagnostica presente sulla riga corrente, anche quando non c'è documentazione da mostrare
`]d` / `[d`	Salta al prossimo/precedente problema nel buffer (qualsiasi gravità)
`]e` / `[e`	Salta al prossimo/precedente **errore** (ignora avvisi/hint di minore importanza)
`]w` / `[w`	Salta al prossimo/precedente **avviso**

Caso d'uso pratico: stai scorrendo un file e noti tre iconcine rosse nel margine, ma il codice sembra a posto a colpo d'occhio. Premi `]e` per saltare dritto al primo errore, leggi il popup automatico (o premi `<leader>cd` per il dettaglio completo), sistemi, `]e` di nuovo per il prossimo. Non devi scorrere riga per riga cercando dove sia il problema: il segno a margine te lo ha già indicato.

Trouble.nvim: la vista d'insieme

Quando i problemi non sono uno isolato ma tanti sparsi per il progetto (dopo un refactoring, dopo aver tirato giù un aggiornamento, o semplicemente a fine giornata prima di un commit), scorrerli uno a uno con `]e` diventa scomodo. Per questo c'è **Trouble**: apre un pannello dedicato con **l'elenco completo** di ciò che stai cercando, navigabile come una lista.

Tasto	Cosa apre
`<leader>xx`	Tutta la diagnostica di **tutto il progetto** (ogni file con errori/avvisi, raggruppati)
`<leader>xX`	Solo la diagnostica del **buffer corrente**
`<leader>cs`	I simboli del documento corrente (funzioni, classi, variabili), utile come indice/mappa del file
`<leader>cS`	Riferimenti/definizioni LSP in una vista Trouble (alternativa al Picker per lo stesso tipo di risultati)
`<leader>xt`	Tutti i commenti **TODO / FIX / HACK** trovati nel codice di progetto
`<leader>xT`	Solo i commenti **TODO / FIX / FIXME** (un filtro più stretto del precedente)
`]t` / `[t`	Salta al prossimo/precedente commento TODO nel buffer, senza nemmeno aprire il pannello

Dentro il pannello di Trouble: ti muovi con `j` / `k`, premi **Invio** per saltare al problema selezionato (il pannello resta aperto, comodo per lavorare voce per voce), oppure `o` per saltarci e chiudere subito il pannello.

Caso d'uso pratico: hai appena finito una sessione di refactoring corposa e prima di fare commit vuoi essere sicuro di non aver lasciato nulla a metà. Premi `<leader>xx`: si apre l'elenco completo di ogni errore rimasto in ogni file del progetto, raggruppato per file. Li risolvi uno per uno con Invio, senza dover ricordare quali file avevi toccato. A parte finito, `<leader>xt` per un ultimo controllo: hai lasciato qualche `// TODO: sistemare questo` dimenticato per strada? Eccolo, in un elenco unico.

7. Completamento Intelligente: blink.cmp e Codeium

Mentre scrivi codice, due sistemi lavorano insieme e in modo trasparente per suggerirti cosa scrivere: **blink.cmp** (il motore di completamento, che pesca da LSP, snippet e buffer aperti) e **Codeium** (il motore AI, che suggerisce interi blocchi di codice basandosi sul contesto). Non devi scegliere quale dei due usare: appaiono **entrambi nello stesso posto**, integrati.

Cosa vedi mentre scrivi

Mentre digiti, iniziano a succedere due cose contemporaneamente:

1. **Il menu di completamento**: un piccolo popup con l'elenco dei possibili completamenti (funzioni, variabili, proprietà, snippet, e — con priorità alta, in cima — i suggerimenti di Codeium, riconoscibili da un'icona dedicata). Ogni voce può mostrare a fianco un'anteprima della documentazione, se disponibile.
2. **Il "ghost text"**: un suggerimento dell'AI mostrato **in grigio, direttamente inline nel testo**, come se stessi scrivendo tu stesso — ma non è ancora stato scritto davvero, è solo un'anteprima "fantasma" di ciò che Codeium propone come continuazione naturale della riga (o di più righe).

Come accetti un suggerimento

Tasto	Cosa fa, in ordine di priorità
`<Tab>`	Se sei dentro uno snippet attivo, avanza al prossimo placeholder. Altrimenti, se c'è un suggerimento AI (ghost text) visibile, **lo accetta per intero**. Altrimenti, si comporta come un Tab normale (indentazione).
`<CR>` (Invio) o `<C-y>`	Conferma la voce selezionata nel **menu** a tendina di completamento (diverso dal ghost text: questo è per quando hai il popup aperto e ci hai navigato sopra)
`<C-n>` / `<C-p>` o frecce	Scorri su e giù tra le voci del menu di completamento

In pratica, **"si sente" così**: cominci a scrivere il nome di una funzione, il menu si apre da solo mostrando le opzioni disponibili nel contesto (variabili nello scope, metodi dell'oggetto, proprietà); se in quel momento Codeium ha già intuito l'intera riga (o le prossime righe) che stai per scrivere, la vedi comparire in anteprima grigia subito dopo il cursore — e con un solo **Tab** la fai diventare codice vero, scritto per intero, invece di digitarla parola per parola. Se invece il suggerimento AI non è quello che volevi, ignoralo semplicemente continuando a digitare (sparisce e si aggiorna da solo), oppure scegli manualmente un'altra voce dal menu con le frecce e Invio.

Caso d'uso pratico: inizi a scrivere una funzione che hai già scritto in modo simile altrove nel progetto (per esempio un altro gestore di richieste HTTP con la stessa forma). Dopo la firma della funzione, Codeium spesso intuisce già l'intero corpo e te lo mostra come ghost text grigio su più righe: un `<Tab>` e te lo ritrovi scritto, pronto per essere adattato al caso specifico, invece di ridigitarlo identico da zero.

Se vuoi autenticare o gestire manualmente Codeium (per esempio al primissimo utilizzo), il comando è `:Codeium Auth`.

---

8. Manutenzione del Sistema

Lazy.nvim: la gestione dei plugin

Tutti i plugin di questa configurazione sono gestiti da **lazy.nvim**, che ha una sua interfaccia testuale completa.

Comando	Azione
`:Lazy` (o `<leader>l`)	Apre la UI principale: stato di ogni plugin, quali sono aggiornabili, tempi di caricamento
`:Lazy sync`	Installa i plugin mancanti, aggiorna quelli esistenti, e **rimuove automaticamente** quelli non più necessari
`:Lazy update`	Solo aggiornamento, senza toccare nient'altro
`:Lazy restore`	Riporta ogni plugin esattamente alla versione registrata in `lazy-lock.json` (utile per tornare a uno stato "noto buono" se un aggiornamento ha rotto qualcosa)
`:LazyExtras`	Il pannello per abilitare/disabilitare intere funzionalità linguistiche precostituite (per esempio, il supporto a un nuovo linguaggio) con un tasto

Dentro la UI di `:Lazy`, premi `U` per aggiornare tutto quello che ha un aggiornamento disponibile, oppure naviga fino a un plugin specifico e premi `Invio` per vedere il suo changelog prima di decidere.

Mason: gli strumenti esterni (LSP, linter, formatter)

Mentre Lazy gestisce i *plugin di Neovim* (scritti in Lua), **Mason** gestisce gli **strumenti esterni**: i veri e propri programmi eseguibili che fanno il lavoro pesante dietro le quinte — i language server (come Pyright o vtsls), i linter (come Ruff o `shellcheck`), i formatter (come `stylua`), e così via.

Comando	Azione
`:Mason` (o `<leader>cm`)	Apre la UI di Mason: elenco di tutto ciò che è installato, cosa è disponibile, e lo stato (installato / non installato / aggiornabile)
`:MasonInstall <nome>`	Installa un tool specifico da riga di comando
`:MasonUninstall <nome>`	Rimuove un tool

Dentro la UI di Mason, i pacchetti installati sono contrassegnati con un'icona verde, quelli disponibili ma non installati con un'icona diversa. Puoi navigare fino a un pacchetto (per esempio `flake8` o `stylua`) e premere `i` per installarlo, `X` per rimuoverlo, `u` per aggiornarlo — tutto senza uscire da Neovim né toccare un terminale esterno.

Caso d'uso pratico: vuoi verificare che `flake8` sia davvero installato e funzionante prima di affidarti al suo linting su un progetto Python importante. Apri `:Mason`, cerchi `flake8` nell'elenco (puoi digitare `/flake8` per filtrare), e vedi a colpo d'occhio se è marcato come installato o meno; se manca, un `i` con il cursore sopra la voce lo installa all'istante.

Il comportamento silenzioso sui file Markdown

Quando apri un file `.md`, noterai due cose rispetto a un file di codice qualsiasi:

1. **Nessuna diagnostica**: niente segni a margine, niente sottolineature, anche se il file contiene righe molto lunghe, indentazioni irregolari o altre cose che un linter Markdown molto rigoroso normalmente segnalerebbe. Questo per un motivo pratico: la scrittura in Markdown (README, note, documentazione) è spesso più libera e informale della scrittura di codice, e le regole di stile tipiche di un linter Markdown risulterebbero solo fastidiose durante la stesura.
2. **Nessuna formattazione automatica al salvataggio**: salvando un file Markdown con `<C-s>` o `:w`, il contenuto resta esattamente come lo hai scritto, senza che un formatter automatico intervenga a riformattarlo a modo suo.

Se in un caso specifico vuoi comunque **formattare manualmente** un file Markdown secondo le regole standard, resta sempre disponibile il comando universale `<leader>cf` (Format), che puoi lanciare quando vuoi, buffer per buffer, restando tu a decidere il momento giusto invece che subirlo a ogni salvataggio.